

EECS 498/CSE 598: Zero-Knowledge Proofs

Winter 2026

Lecture 19

Paul Grubbs

paulgrub@umich.edu

Beyster 4709

Agenda for this lecture

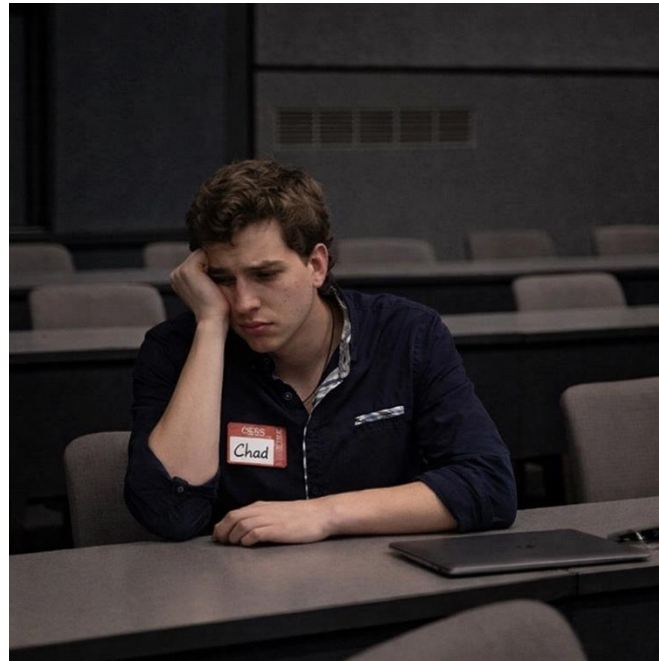
- Announcements
- Non-native arithmetic: the small-modulus case
- Non-native arithmetic: the large-modulus case
- Composite proofs
- Parsing and string operations
- Other issues and applications

Agenda for this lecture

- **Announcements**
- Non-native arithmetic: the small-modulus case
- Non-native arithmetic: the large-modulus case
- Composite proofs
- Parsing and string operations
- Other issues and applications

Announcements

- The plan is to release P4 and theory pset 2 ASAP.
 - Note there will be no P5 nor a final exam. These will be the last assignments of the semester.
- 598 research track: **project proposal due Friday 3/27**
- All work for this course must be completed by our final exam date, April 27th
- Reminder: you can get participation credit for going to discussion.
 - Chad is getting lonely on Fridays!



Agenda for this lecture

- Announcements
- **Non-native arithmetic: the small-modulus case**
- Non-native arithmetic: the large-modulus case
- Composite proofs
- Parsing and string operations
- Other issues and applications

Motivation

- I know an RSA signature σ on message m
- This HTTP header matches a network policy
- This regex matches a private document
- I have a signed JSON blob from the DMV that says I am over 18.
- For a hash output s , I know an $x \in F$ such that $s = H(xG)$

Motivation

Computation with larger integers than normal
R1CS (RSA modulus ≈ 2048 bits)

- I know an RSA signature σ on message m
- This HTTP header matches a network policy
- This regex matches a private document
- I have a signed JSON blob from the DMV that says I am over 18
- For a hash output s , I know an $x \in F$ such that $s = H(xG)$

Parsing, extracting values
from structured data

Mixed-arithmetic computation

Motivation

Computation with larger integers than normal
R1CS (RSA modulus ≈ 2048 bits)

- I know an RSA signature σ on message m
- This HTTP header matches a network policy
- This regex matches a private document
- I have a signed JSON blob from the DMV that says I am over 18
- For a hash output s , I know an $x \in F$ such that $s = H(xG)$

Parsing, extracting values
from structured data

Mixed-arithmetic computation

Three strategies for computations like this:

1. Encode non-native arithmetic in R1CS
2. Build “composite” proofs that span different proof systems
3. Efficiently represent string operations

What is "non-native" arithmetic?

= No precise def'n

prove computation that's not mod- p
p our RICS modulus

- Small modulus - $m \ll p$ [e.g. $m = 2^{32}$]
- Large modulus - $m \gg p$ [e.g. $m \approx 2^{98}$ bit]
- "Other" (Field extension)
 $GF(2^{128})$

Small modulus arithmetic in R1CS

Since $m \ll p$, can treat mod- p arith as integer arithmetic

E.g. $C = a + b \bmod 2^{32}$

Since a, b at most 32 bits, $a+b$ at most 33

$$\rightarrow a + b = C + K 2^{32} \quad \text{and } K \in \{0, 1\}$$

Range checks!

Need to range check $a, b, C \in [0, 2^{32}-1]$

Range check cost $\approx 99\%$

hookup arguments



Small modulus arithmetic in R1CS

Agenda for this lecture

- Announcements
- Non-native arithmetic: the small-modulus case
- **Non-native arithmetic: the large-modulus case**
- Composite proofs
- Parsing and string operations
- Other issues and applications

The large-modulus case

Can I treat mod- p arithmetic as integers

Limb-wise arithmetic

Add-subtract mod m :

How big are the limbs?

limb-wise arith. with carry

How to multiply?

$$X \cdot Y = Z \pmod{m}$$

XJSNARK: linear-time mod RLS

view limbs of X, Y, Z as univariate polys [d limbs = $d-1$ degree]

To compute $X \cdot Y = Z$

Evaluate X, Y, Z at $2d$ points $c \in \{0, \dots, 2d-1\}$

$$X(c) \cdot Y(c) = Z(c) \quad \forall c$$

Mod-reduce Z : give q, r ,

$$\text{check } Z = qM + r \quad r < M$$

Goal: mod- m arith.

$\lceil \log m \rceil$ = bit length of ints in program

we have: mod- p arith (RLCS)

$\lceil \log p \rceil$ = bit-length of witness values

Agenda for this lecture

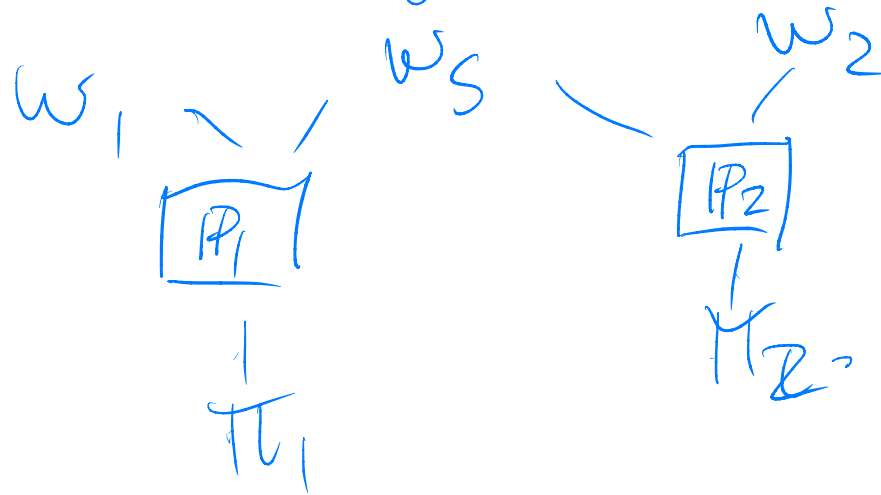
- Announcements
- Non-native arithmetic: the small-modulus case
- Non-native arithmetic: the large-modulus case
- **Composite proofs**
- Parsing and string operations
- Other issues and applications

Composite proofs

Basic idea: for mixed-arithmetic computations, split into subcircuits and prove each in a “natural” way

$$\mathcal{R} = \{ (S, G; X) : S = H(X, G) \}$$

Shared witness
↓



Cheap consistency check

Agenda for this lecture

- Announcements
- Non-native arithmetic: the small-modulus case
- Non-native arithmetic: the large-modulus case
- Composite proofs
- **Parsing and string operations**
- Other issues and applications

Parsing and string operations

String operations in increasing order of difficulty:

1. Substring containment → Prove substring occurs
2. Regex match/non-match → Run DFA/NFA on committed data
3. Extract a single field from a structured data object
4. Fully parse a string w/r/t some grammar

"Context integrity"

→ "Partial" parse

↳ "Fancy"

5 Convert data representation
(Hex / Base64 : lookups)

Proving DFA executions

Committed doc



Run DFA

Set of states

transition function

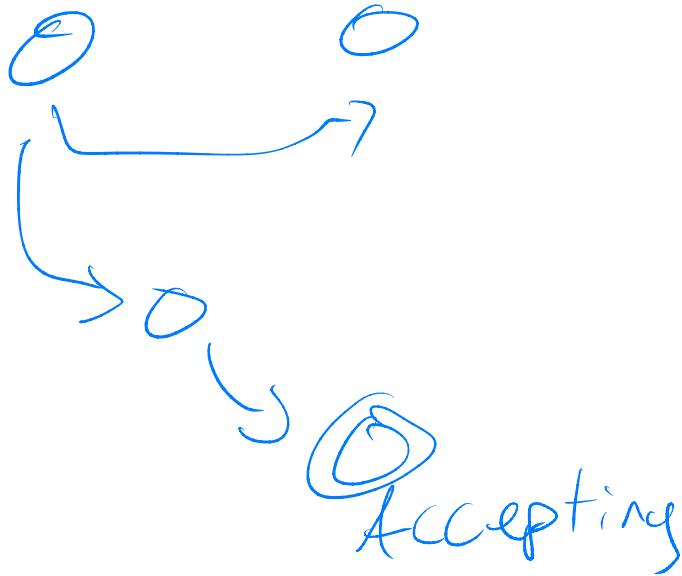
Encode transition t^n

as big OR

$\gamma(st, c)!$

IF $st = i$ and $c = 's'$

$st' = j$



Agenda for this lecture

- Announcements
- Non-native arithmetic: the small-modulus case
- Non-native arithmetic: the large-modulus case
- Composite proofs
- Parsing and string operations
- Other issues and applications

Lecture takeaways

Representation matters!